

CS F407 – Bayesian Networks Lab: Autoregressive Language Models

This report presents the solution to the laboratory exercise [BN_lab.pdf](#).

File	Contents
bn_language_model.py	First-order and second-order autoregressive models (one <code>NGramModel</code> class with an <code>order</code> parameter), tests, generation, and comparison
results.txt	Full output of <code>python bn_language_model.py</code> ; every number in this report is taken from it
generated_first_order.txt , generated_second_order.txt	The 20 sampled sentences from each model
README.md / REPORT.pdf	This report, in Markdown and PDF form

Requirements: Python 3 (standard library only). Sampling uses a fixed seed (`random.Random(407)`), so every result is reproducible:

```
python bn_language_model.py
```

Part I – From probability to language

Question 1. Why is the autoregressive decomposition useful for generating text? The chain rule rewrites the joint distribution over a whole sentence, which is intractable to represent directly, as a product of *one-step* conditionals $P(X_t \mid X_1, \dots, X_{t-1})$. Each factor is a distribution over a single next word, which is small enough to estimate and to sample from. Generation then becomes a sequential procedure:

1. sample X_1 ;
2. condition on it and sample X_2 ;
3. continue in the same way until an end token is produced.

The procedure is exact: sampling each factor in turn yields a sample from the full joint distribution. It also matches the left-to-right order in which text is written, so the same factorisation serves for scoring a sentence (multiply the factors) and for producing one (sample the factors).

Part II – A Bayesian network for text

Question 2. What independence assumption does the chain $X_1 \rightarrow X_2 \rightarrow X_3 \rightarrow \dots$ make? It is the first-order Markov assumption: given the immediately preceding word, the next word is conditionally independent of all earlier words.

$P(X_t \mid X_1, \dots, X_{t-1}) = P(X_t \mid X_{t-1})$, equivalently $X_t \perp \{X_1, \dots, X_{t-2}\} \mid X_{t-1}$.

In the graph, X_{t-1} is the only parent of X_t , and it *d-separates* X_t from every earlier node.

Part III – Dataset

The six sentences of the handout were lower-cased, split into word tokens, and wrapped in markers, e.g. `<START> the cat sat on the mat <END>`. The vocabulary contains 10 words: cat, dog, mat, on, park, ran, rug, sat, the, to.

Part IV – Conditional probability table (first-order)

$$P(w_j | w_i) = C(w_i, w_j) / \sum_k C(w_i, w_k):$$

Current word	Counts of next word	P(next current)
<START>	the: 6	the 1.000
the	cat 3, dog 3, mat 2, rug 2, park 2	cat 0.250, dog 0.250, mat 0.167, park 0.167, rug 0.167
cat	sat 2, ran 1	sat 0.667, ran 0.333
dog	sat 2, ran 1	sat 0.667, ran 0.333
sat	on 4	on 1.000
ran	to 2	to 1.000
on	the 4	the 1.000
to	the 2	the 1.000
mat / rug / park	<END>	<END> 1.000

Question 3 – zero-probability transitions. Every pair not observed in the training data receives probability 0. For the five required words:

Context	Next words with probability 0
the	on, ran, sat, the, to, <END>
cat	cat, dog, mat, on, park, rug, the, to, <END>
dog	cat, dog, mat, on, park, rug, the, to, <END>
sat	everything except <i>on</i> (e.g. $P(\text{to} \text{sat}) = 0$)
ran	everything except <i>to</i> (e.g. $P(\text{on} \text{ran}) = 0$)

Most of these zeros are linguistically reasonable ("the the", "cat dog"). Some, however, rule out perfectly acceptable English, such as "sat by" or "ran on", merely because those phrases did not occur in six sentences. Maximum-likelihood counting cannot distinguish *impossible* from *unobserved*.

Part V – LLM implementation

LLM used: Claude (Anthropic).

Prompt (as suggested in the handout):

Write a simple Python implementation of a first-order autoregressive language model. The model should: 1. take a list of tokenised sentences as training data; 2. count transitions between consecutive tokens; 3. construct the conditional distribution $P(X_t | X_{t-1})$; 4. display the probabilities for a specified previous token; 5. predict the most probable next token; 6. generate a sentence by repeatedly sampling the next token; 7. stop when the $\langle \text{END} \rangle$ token is generated. Do not use a machine-learning library or a pretrained language model. Use ordinary Python data structures and random sampling.

Part VI – Inspection of the generated code

Question 4. Where are the transition counts stored? In `self.counts`, a `defaultdict(Counter)` that maps a context tuple to a `Counter` of next tokens (lines 44–49 of `bn_language_model.py`). For example, `counts[("the",)][ "cat"] == 3`.

Question 5. Where is $P(X_t | X_{t-1})$ computed? In the dictionary comprehension that builds `self.cpt` (line 51). Each count is divided by the total count for its context: `c / sum(nexts.values())`. The method `distribution()` returns the resulting table for a given context.

Question 6. How is the next word chosen? Both options are implemented, selected by the `mode` argument of `generate()`:

- `most_probable()` (line 65) always returns $\arg \max_v P(v | \text{context})$, with ties broken alphabetically. This is **greedy** and deterministic.
- `sample_next()` (line 71) draws the next word at random with probabilities equal to the CPT entries, using `rng.choices(words, weights=...)`. This is **sampling**.

The difference: greedy selection always follows the single most likely continuation, so it produces the same output every time and never produces lower-probability words. Sampling reproduces the model's full distribution, so a word with probability 0.167 appears in about one sixth of draws.

Question 7. What happens for a word with no observed transitions? Such a context has no CPT row, so $P(X_t | w)$ is undefined. The implementation raises an explicit `KeyError` rather than returning an empty or arbitrary distribution:

```
KeyError: "context ('bird',) never observed in training data; P(X_t | ('bird',)) is undefined"
```

This is a deliberate correction. Because `counts` is a `defaultdict`, a direct lookup of an unseen word would silently *create* an empty entry, and sampling from it would then fail with an obscure `IndexError`. Lookups therefore go through the ordinary dictionary `cpt`. In practice the problem is addressed by *smoothing* (e.g. add-one or back-off), which assigns a small non-zero probability to unseen events.

Part VII – Testing the probability model

The invariant $\sum_v P(v | w) = 1$ was checked for **every** context of both models:

Model	Contexts checked	Minimum total	Maximum total	Failures
First-order	11	1.0000000000000000	1.0000000000000000	0
Second-order	15	1.0000000000000000	1.0000000000000000	0

Question 8. What would a total of 0.87 indicate? The distribution for that context is not normalised, so the implementation is incorrect. Possible causes are:

- a normalising denominator that includes counts from other contexts;
- some next tokens dropped from the table, e.g. ⟨END⟩ excluded from the row but included in the total;
- counting errors at sentence boundaries;
- a bug in smoothing that removes probability mass without redistributing it.

The missing 0.13 of probability mass means the sampler, or any score computed from the table, no longer matches the intended model.

Part VIII – Next-word prediction

Context w	$P(X_{t+1} X_t = w)$	arg max
the	cat 0.250, dog 0.250, mat 0.167, park 0.167, rug 0.167	cat (tied with dog; alphabetical tie-break)
cat	sat 0.667, ran 0.333	sat
dog	sat 0.667, ran 0.333	sat
sat	on 1.000	on
ran	to 1.000	to
on	the 1.000	the
to	the 1.000	the

Question 9. Are the most probable predictions the ones a person would expect? Only partly. After "sat" the model predicts "on", which is what a reader would expect. After "the", however, the model's answer is "cat", and it is independent of whether "the" opens the sentence or follows "on". A human reader who has just seen "sat on the" strongly expects a surface such as *mat* or *rug*, not *cat*. The model cannot represent this expectation, because it conditions on one word only.

The model also assigns probability 1 to "sat → on" and 0 to "sat by", which a person knows to be possible. A probability model reflects exactly the statistics of its training data, under the independence assumptions of its structure. Human linguistic expectation draws on far more context, world knowledge, and grammatical competence.

Part IX – Generated text (first-order, sampling, 20 sentences)

#	Sentence	In training data?	P under the model
1	the cat ran to the dog ran to the rug	No	0.00116
2	the park	No	0.167
3	the cat sat on the mat	Yes	0.0278
4	the dog ran to the mat	No	0.0139
5	the dog ran to the rug	No	0.0139
6	the dog sat on the dog sat on the mat	No	0.00463
7	the cat sat on the mat	Yes	0.0278
8	the mat	No	0.167
9	the cat ran to the dog sat on the cat sat on the rug	No	0.000386
10	the park	No	0.167
11	the cat sat on the cat sat on the dog sat on the mat	No	0.000772
12–14	the rug (×3)	No	0.167
15	the dog ran to the dog sat on the dog ran to the dog sat on the rug	No	3.22×10^{-5}
16	the rug	No	0.167
17	the park	No	0.167
18	the mat	No	0.167
19	the rug	No	0.167
20	the mat	No	0.167

The sentences are saved in `generated_first_order.txt`. Two-word "sentences" such as *the rug* are the most frequent outputs (11 of 20 samples). This is expected, because they are the most probable sentences under the model:

$$P(\textit{the rug}) = P(\textit{the} \mid \langle \text{START} \rangle) \cdot P(\textit{rug} \mid \textit{the}) \cdot P(\langle \text{END} \rangle \mid \textit{rug}) = 1 \times 1/6 \times 1 = 0.167.$$

The mat and *the park* have the same probability. The chain has no memory that *rug* only ever followed "on the", so it treats "the → rug → ⟨END⟩" as a legal path.

Part X – Greedy versus sampling

Mode A (greedy), 5 runs – all identical:

the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on ...[truncated at MAX_LEN]

Mode B (sampling), 5 runs:

the dog sat on the dog sat on the park
the mat
the mat
the dog ran to the dog ran to the park
the mat

Question 10. Which mode produces more variation, and why? Sampling. Greedy decoding is a deterministic function of the context, so it produces the same sentence on every run. Here that sentence never ends: the arg max after "the" is "cat" rather than an end-bearing word such as "mat", so greedy decoding cycles through "the cat sat on the" indefinitely. A length limit (`MAX_LEN = 20`) was therefore required to stop it; without that guard the generator would not terminate. Sampling draws each word in proportion to its probability, so different runs follow different branches of the chain and produce different sentences, including ones that terminate.

Part XI – A second-order Bayesian network

The second-order model estimates $P(X_t | X_{t-2}, X_{t-1})$; each sentence is padded with two `<START>` tokens. Selected rows of its CPT:

Context (X_{t-2}, X_{t-1})	$P(X_t \text{context})$
<code><START>, <START></code>	the 1.000
<code><START>, the</code>	cat 0.500, dog 0.500
the, cat	sat 0.667, ran 0.333
the, dog	sat 0.667, ran 0.333
cat, sat	on 1.000
dog, ran	to 1.000
on, the	mat 0.500, rug 0.500
to, the	park 1.000
cat, on (unseen)	undefined, reported as a <code>KeyError</code>

The two highlighted rows show the benefit of the additional context. The first-order model mixed every continuation of "the" together, whereas the second-order model knows that "on the" is followed by a surface and "to the" by a destination.

Question 11. How does the second-order model differ from the first-order model?

1. **Graph structure.** Each X_t has two parents, X_{t-2} and X_{t-1} , instead of one. The graph therefore gains the edges $X_{t-2} \rightarrow X_t$ and X_t is conditionally independent of the past only given *both* preceding

words.

2. **Conditional probability table.** Its rows are indexed by *pairs* of words rather than single words. The number of possible rows grows from 11 to 111 in this vocabulary; in general it grows from V to about V^2 .
3. **Context available.** Two preceding words instead of one, which is enough to distinguish "on the" from "to the" and "the cat" from "cat sat".
4. **Data needed.** Many more contexts must each be observed often enough for their rows to be estimated reliably. With the same six sentences, 96 of the 111 possible contexts are never seen at all.

Part XII – Second-order implementation with the LLM

Prompt used:

Modify the existing first-order autoregressive model into a second-order model. The model should estimate $P(X_t | X_{t-2}, X_{t-1})$. Represent the model using counts of observed triples and use these counts to construct conditional probability distributions. Do not replace the model with a neural network or a pretrained language model.

What should change, stated before accepting the code:

- contexts become 2-tuples;
- sentences need two $\langle \text{START} \rangle$ pads, so that $P(X_1)$ and $P(X_2 | X_1)$ are represented;
- the counts become counts of triples;
- normalisation is per pair;
- generation must shift a two-word window.

Nothing else should change. The accepted implementation generalises the class with an `order` parameter, so both models share the same counting, normalisation, sampling, and validation code. The normalisation test (Part VII) passed for all 15 observed second-order contexts.

Part XIII – Comparison of the two models

Measure	First-order	Second-order
Possible contexts	11	111
Observed contexts	11	15
Zero-probability (unseen) contexts	0	96
Free parameters of the full CPT	110	1,110
Non-zero probabilities actually estimated	17	19
Distinct sentences in 20 samples	11	6
...of which not in the training data	10	0

Qualitative coherence.

- **First-order:** the model is creative but often incoherent. It produces two-word "sentences" (*the mat, the park*), semantically odd combinations (*the dog ran to the mat*), and long repetitive loops (*the cat ran to the dog sat on the cat sat on the rug*). Each adjacent pair is locally plausible, but the sentence as a whole is not.
- **Second-order:** every sample is fluent and well formed (e.g. *the cat ran to the park, the dog sat on the rug*). However, all six distinct outputs are sentences from the training data. With six training sentences and two words of context, the model has effectively **memorised** the corpus: every observed context has only the continuations seen in training, so no new sentence can be produced.

Question 12. Why can more context improve prediction yet make the model harder to estimate?

More context removes ambiguity: $P(\cdot \mid \text{on, the})$ places all its mass on {mat, rug}, whereas $P(\cdot \mid \text{the})$ spreads it over five words, so predictions become sharper and more coherent. The cost lies in the size of the CPT. The number of possible contexts grows as V^n for an n -th order model (here from 11 to 111 contexts, and from 110 to 1,110 free parameters), while the amount of training data stays fixed. Each context is therefore observed fewer times; most contexts (96 of 111) are never observed; and the rows that are observed are estimated from one or two examples, which leads to overfitting. This is the bias–variance trade-off, and it is the reason modern language models replace explicit tables with a parametrised function that *shares* statistical strength across contexts.

Part XIV – Connection to modern language models

A modern autoregressive language model optimises the same objective,

$$P(x_1, \dots, x_T) = \prod_t P(x_t \mid x_1, \dots, x_{t-1}),$$

but represents each conditional distribution with a neural network rather than a table. The network maps a long context to a probability vector over the vocabulary. As the handout's comparison table notes, the models differ in representation (CPT vs. network), context (a fixed window vs. a long learned context), parameters (explicit probabilities vs. learned weights), and learning (counting vs. gradient-based training). They share the generation procedure: sampling one token at a time from $P(\text{next token} \mid \text{previous tokens})$.

Part XV – Reflection on the role of the LLM

Question 13. Why is Approach B preferable to Approach A? Approach A ("Write a Python language model for me") leaves the model undefined. The LLM may return anything from a bigram counter to a call to a pretrained network, and there is then no specification against which to check the result. Approach B names the probabilistic model exactly, $P(X_t \mid X_{t-1})$ estimated from transition counts with sampling-based generation. In particular it allows the engineer to:

- **specify the intended behaviour** in advance, so that the generated code can be judged correct or incorrect rather than merely plausible;
- **understand the representation:** knowing that the model is a table of conditional probabilities makes it possible to find where the counts and probabilities live in the code (Questions 4 and 5);
- **validate the implementation:** the specification yields concrete checks, such as reproducing the hand-computed $P(\text{cat} \mid \text{the}) = 0.25$;

- **test probabilistic invariants**, such as every CPT row summing to 1, which hold only if the code implements the stated model;
- **distinguish implementation from model**: whether generation is greedy or sampled, or how unseen contexts are handled, are implementation decisions that should be made deliberately rather than inherited silently from the LLM's choices.

Example of inspected and corrected LLM-generated code. Two problems were found during inspection:

1. **Non-terminating greedy decoding.** The generation loop stopped only when `<END>` was produced. Greedy decoding on this corpus never produces `<END>`, because it cycles through "the cat sat on the" indefinitely. A maximum length (`MAX_LEN`) was added, and truncated outputs are marked explicitly.
2. **Unseen contexts.** Looking up an unseen word through the `defaultdict` of counts silently inserts an empty row, which later causes an unexplained `IndexError` during sampling. Lookups were redirected to the plain CPT dictionary, which raises a descriptive `KeyError` instead (Question 7).

Final question

Question 14. What did viewing the language model as a Bayesian network contribute?

- **A representation of dependencies.** The graph states explicitly which previous words each word depends on, one parent for the first-order model and two for the second-order model. This is exactly the design decision that separates the two models.
- **A factorisation of the joint distribution.** The network yields the product $P(X_1) \prod P(X_t \mid \text{parents}(X_t))$ directly. It reduces an intractable joint table to small local CPTs, and it gives a way to score whole sentences (e.g. $P(\text{the cat sat on the mat}) = 0.0278$ under the first-order model).
- **A principled method of generation.** *Ancestral sampling*, i.e. sampling each node given its already-sampled parents in topological order, is precisely the left-to-right generation procedure used in Part IX.
- **A way to reason about independence assumptions.** The first-order network makes explicit that $X_t \perp X_{t-2} \mid X_{t-1}$. This assumption explains the failures observed: the model cannot distinguish "on the " *from* "to the ", and it generates *the mat* as a complete sentence.
- **A way to understand the effect of increasing context.** Adding a parent enlarges each CPT multiplicatively ($11 \rightarrow 111$ contexts). This explains why the second-order model is more coherent but largely unseen and prone to memorisation.
- **A way to test an implementation against its specification.** Each node's CPT must be a valid conditional distribution, which gave the normalisation test of Part VII. Each CPT entry has a closed-form estimate, which can be checked by hand (Part IV).